Topic Proposal: Modelos de Concorrência num Microserviço Java

Pedro Gomes
Programação Paralela e Concorrente
Mestrado em Engenharia Informática
Faculdade de Ciências, Universidade de Lisboa
fc67991@alunos.ciencias.ulisboa.pt

Abstract

Este estudo compara platform threads, virtual threads e I/O assíncrono num microserviço Java com chamadas HTTP a serviços simulados. Avaliam-se throughput, latência e consumo de memória sob diferentes níveis de concorrência e características de carga, identificando as condições em que cada modelo apresenta melhor desempenho.

1 Topic and Motivation

Serão comparados três modelos de concorrência num microserviço Java: chamadas bloqueantes com platform threads, chamadas bloqueantes com virtual threads e chamadas assíncronas com `CompletableFuture`. Como os microserviços passam frequentemente tempo à espera de respostas externas, importa perceber como o modelo de concorrência afeta o desempenho sob carga. Experiências controladas ajudam a escolher o modelo certo para serviços dominados por I/O.

2 Research Questions

RQ1. Como variam o throughput e a latência dos três modelos de concorrência com o aumento dos pedidos concorrentes?
H1. Sob elevada concorrência e carga dominada por I/O, espera-se que as virtual threads e o I/O assíncrono superem um conjunto fixo de platform threads em throughput e latência de cauda. Resultados semelhantes ou inferiores contrariariam esta hipótese.
RQ2. Como influenciam a latência dos serviços externos, o número de chamadas HTTP por pedido e o trabalho de CPU o desempenho relativo dos três modelos?
H2. Espera-se que as vantagens das virtual threads e do I/O assíncrono diminuam à medida que aumenta a proporção de trabalho de CPU.

3 Type of Study

(c) Extensão de estudos existentes. O estudo [1] compara virtual threads e platform threads em Spring Boot e Helidon, enquanto este mantém a framework fixa e compara três modelos de concorrência, incluindo I/O assíncrono com CompletableFuture. Em relação aos estudos existentes, varia também a latência dos serviços simulados, o número de chamadas HTTP e o trabalho de CPU por pedido, permitindo avaliar o impacto destas características no desempenho relativo dos modelos.

4 Planned Methodology

System(s) under study. Será desenvolvido um microserviço HTTP com Spring Boot, responsável por agregar respostas de serviços HTTP simulados (stubs). Serão implementadas três variantes: (1) conjunto fixo de threads de plataforma com chamadas bloqueantes; (2) uma thread virtual por tarefa, com chamadas bloqueantes; e (3) chamadas assíncronas através de HttpClient.sendAsync() e CompletableFuture.Não é usado framework reativa, para medir o modelo e não a framework.
Independent variables. Modelo de concorrência; número de clientes concorrentes (10,100,1000,2000); latência dos serviços externos (fixa, variável e cauda longa); número de chamadas por pedido (1,3, 5); e carga de CPU por pedido (nenhum, moderado, alto).
Dependent variables. Throughput, latência (p50, p95 e p99), consumo de memória e taxa de erros e timeouts. Um gerador de carga externo recolherá as estatísticas dos pedidos, enquanto ferramentas da JVM e do sistema operativo monitorizarão a memória.
Held constant. Hardware, versão do Java, cliente HTTP, payload, lógica de processamento e configuração de ligações. Gerador de carga noutra máquina. Serão realizados warm-ups e repetições (a justificar no paper).

5 Experimental Setup and Feasibility

O microserviço será executado num portátil com macOS, Java 21 e 32 GB de RAM; o gerador de carga correrá noutra máquina. Serão registadas as especificações do hardware e do software.

Os principais riscos são a variabilidade das medições e a competição por recursos. Testes-piloto permitirão validar a metodologia e reduzir o âmbito à RQ1, se necessário.

6 Plan

Até 11 Nov: Implementar as três variantes, os serviços simulados e o gerador de carga; realizar testes-piloto. **Até 2 Dez:** Concluir as medições, analisar resultados, produzir gráficos, discutir ameaças à validade e entregar o artigo e o código de reprodução.

7 Key References

[1] Anja Dundov. 2026. *Evaluating Java Virtual Threads for Concurrent Spring Boot and Helidon Services.* In *2026 49th MIPRO ICT and Electronics Convention (MIPRO)*, Opatija, Croatia. IEEE. https://doi.org/10.1109/MIPRO70003.2026.11591610

[2] V. Pandita. 2024. *Benchmarking the Performance of Java Virtual Threads in High-Throughput Workloads.* MSc thesis, National College of Ireland. https://norma.ncirl.ie/8134/

Use of AI tools. Claude (Anthropic): brainstorming do tema, discussão do desenho experimental, pesquisa de literatura e revisão desta proposta.